Java Backend Architecture

Interview Series

Chapter 15.2
Service Decomposition & Bounded Contexts

50+ Interview Q&A | Code Examples | Architecture Diagrams | Advanced Topics

Chapter 15.2 – Service Decomposition & Bounded Contexts

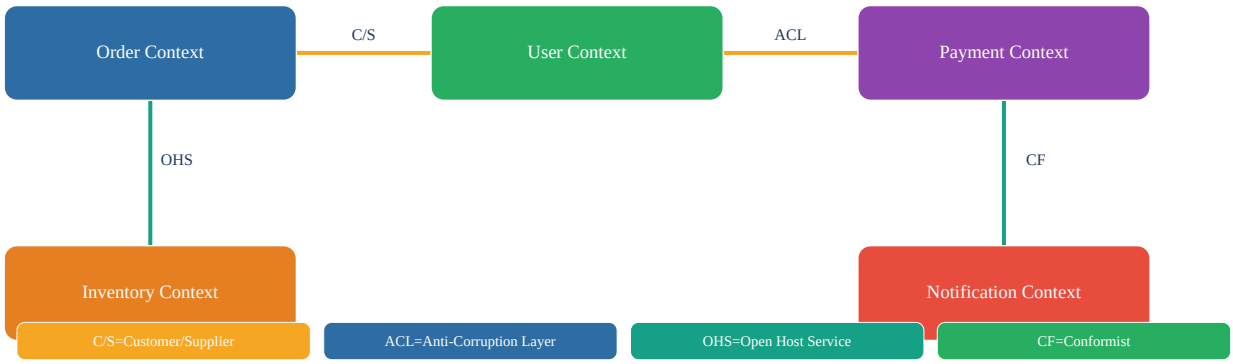
Introduction

Domain-Driven Design (DDD) provides the conceptual tools for decomposing complex systems into well-bounded services. Bounded Contexts define explicit model boundaries where domain terms have specific, unambiguous meanings. Context Maps capture the relationships between contexts. Getting decomposition right is the most important—and hardest—step in microservices design. Wrong boundaries create distributed monoliths; correct boundaries enable true autonomy.

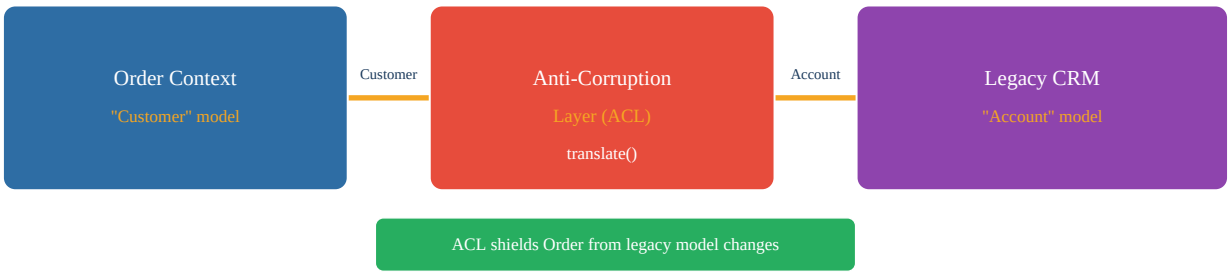
Key Definitions

Bounded Context	Explicit boundary within which a domain model is defined and applicable. Terms have precise meanings inside the context. E.g., "Customer" means different things in Sales vs. Support contexts.
Context Map	Visual/documented representation of all bounded contexts in a system and their integration relationships (patterns like ACL, C/S, Conformist, Shared Kernel).
Shared Kernel	Two contexts share a subset of the domain model. Both teams must agree on all changes. High coordination cost—use sparingly.
Customer/Supplier	Upstream-downstream relationship. Downstream (customer) depends on upstream (supplier). Supplier should accommodate customer needs in planning.
Conformist	Downstream conforms to upstream model with no negotiating power (e.g., integrating with a 3rd-party API you cannot change).
Anti-Corruption Layer	Translation layer protecting a context from an external model. Converts external model to internal domain language. Essential when integrating with legacy systems.
Open-Host Service	Context provides a formal, published protocol (API) for many consumers. Maintains backward compatibility. Example: a public REST API with OpenAPI spec.
Published Language	Shared, well-documented data format for integration (Avro schema, Protobuf .proto, JSON Schema). Enables independent evolution within the schema contract.
Aggregate	Cluster of domain objects treated as a single unit for data changes. Has a root entity (Aggregate Root) that controls access. E.g., Order aggregate contains OrderLines.
Value Object	Immutable object defined by its attributes, not identity. E.g., Money(100, USD). No ID field. Equality by value. Side-effect-free.
Domain Event	Fact that something happened in the domain. Immutable, past tense. E.g., OrderPlaced, PaymentProcessed. Used for integration between bounded contexts.
Ubiquitous Language	Shared language between developers and domain experts within a bounded context. Used in code, conversations, and documentation. Eliminates translation overhead.

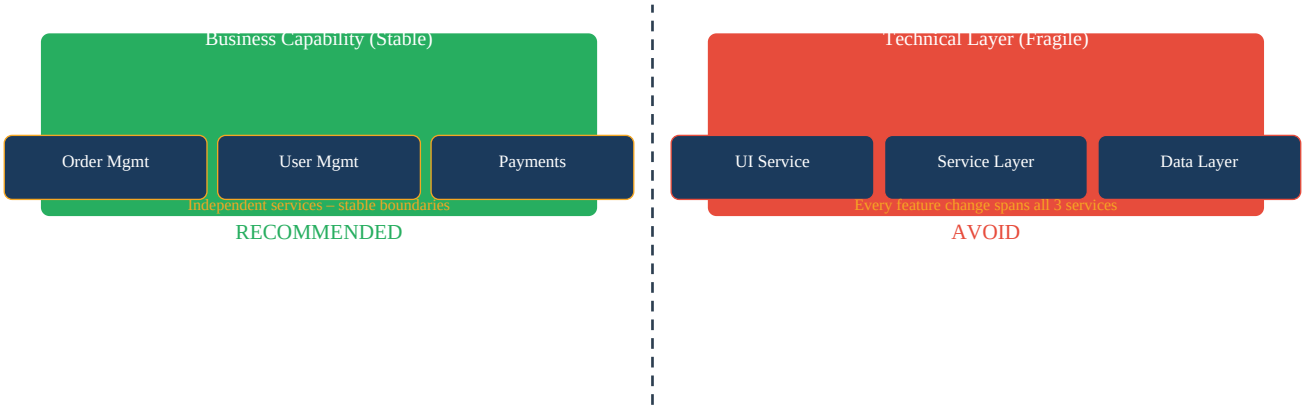
Context Map – Bounded Context Relationships



Anti-Corruption Layer – Model Translation



Business Capability vs Technical Layer Decomposition



Context Map Patterns Reference

Pattern	Relationship	Use When	Risk
Shared Kernel	Both share model subset	Very tight coupling needed	High – both teams must coordinate
Customer/Supplier	Upstream/downstream	Downstream depends on upstream	Medium – upstream must plan for downstream
Conformist	Downstream adapts fully	No negotiating power (3rd party)	Low – but model leaks in

ACL	Translation layer	Legacy/external integration	Low – protected boundary
Open-Host Service	Published protocol	Many consumers, formal API	Low – backward compat required
Published Language	Shared schema/format	Schema-driven integration	Low – schema versioning needed
Separate Ways	No integration	Teams fully independent	Low – duplication acceptable

Aggregate Design in Java

```
// Aggregate Root – Order controls all modifications @Entity public class Order { // Aggregate Root @Id private OrderId id; private CustomerId customerId; // Reference by ID only, never direct object @OneToMany(cascade = CascadeType.ALL, orphanRemoval = true) private List<OrderLine> lines = new ArrayList<>(); // Part of Order aggregate private Money totalAmount; private OrderStatus status; // All mutations go through aggregate root public void addLine(ProductId productId, int qty, Money price) { if (status != OrderStatus.DRAFT) throw new IllegalStateException("Cannot modify confirmed order"); lines.add(new OrderLine(productId, qty, price)); recalculateTotal(); } public List<DomainEvent> confirm() { this.status = OrderStatus.CONFIRMED; return List.of(new OrderConfirmedEvent(id, customerId, totalAmount)); } } // Value Object – immutable, equality by value public record Money(BigDecimal amount, Currency currency) { public Money add(Money other) { if (!currency.equals(other.currency)) throw new IllegalArgumentException(); return new Money(amount.add(other.amount), currency); } }
```

Domain Event – Cross-Context Integration

```
// Domain Event – published when order is confirmed public record OrderConfirmedEvent( OrderId orderId, CustomerId customerId, Money totalAmount, Instant occurredAt ) implements DomainEvent {} // In Order service – publish event after transaction @Service @Transactional public class OrderApplicationService { public void confirmOrder(OrderId orderId) { Order order = orderRepository.findById(orderId).orElseThrow(); List<DomainEvent> events = order.confirm(); orderRepository.save(order); events.forEach(eventPublisher::publish); // Outbox pattern in practice } } // In Inventory service – consume order confirmed event @KafkaListener(topics = "order.confirmed") public void onOrderConfirmed(OrderConfirmedEvent event) { inventoryService.reserveStock(event.orderId(), event.lines()); }
```

Anti-Corruption Layer Implementation

```
// ACL: translate legacy CRM "Account" to our "Customer" @Component public class CrmAntiCorruptionLayer { private final LegacyCrmClient crmClient; public Customer getCustomer(CustomerId id) { // Call legacy system CrmAccount account = crmClient.findAccount(id.value()); // Translate to our domain model return new Customer( new CustomerId(account.getAccountId()), new FullName(account.getFirstName(), account.getLastName()), new Email(account.getEmailAddress()), mapTier(account.getAccountType()) // Different tier models ); } private CustomerTier mapTier(String accountType) { return switch (accountType) { case "PREMIUM_GOLD" -> CustomerTier.VIP; case "STANDARD" -> CustomerTier.REGULAR; default -> CustomerTier.BASIC; }; } }
```

Interview Q&A;

Q1. What is a Bounded Context in DDD?

A Bounded Context is an explicit boundary within which a particular domain model applies. Inside the boundary, all terms of the Ubiquitous Language have precise, unambiguous meanings. The same word ('Customer') can mean different things in different contexts. Services map 1:1 with bounded contexts in microservices architecture.

Q2. How do you identify bounded contexts in a real-world system?

Look for: (1) Places where the same word means different things (e.g., 'product' in catalog vs. order), (2) Teams that work independently and own different parts of the system, (3) Different data lifecycles (order data vs. product data vs. user data), (4) Event storming workshops to map domain events and find natural aggregate clusters.

Q3. What is the difference between an Entity and a Value Object?

Entity: has a unique identity (ID), mutable over time, equality by identity. E.g., Order with orderId. Value Object: no identity, defined entirely by its attributes, immutable, equality by value. E.g., Money(100, USD). In Java, Value Objects are best modeled as records or immutable classes.

Q4. What is an Aggregate Root?

The entry point to an aggregate cluster. All external references must go through the root. All invariants of the aggregate are enforced by the root. Persistence is done at the aggregate root level. Other services reference aggregates only by the root's ID, never by direct object reference.

Q5. Why should you only reference aggregates by ID across service boundaries?

Direct object references would create coupling between aggregates (and services). If Service A holds a reference to an object from Service B's aggregate, any change to that object structure breaks Service A. By ID reference only: each service fetches its own copy of the data it needs.

Q6. Explain the Anti-Corruption Layer pattern.

An ACL is a translation layer between two bounded contexts with incompatible models. It prevents the external model from 'corrupting' the internal domain model. The ACL converts external concepts (e.g., legacy CRM 'Account') to internal domain concepts ('Customer'). Implemented as a service/adapter class that wraps all external system calls.

Q7. When would you use a Shared Kernel?

Only when two closely related contexts genuinely share core concepts that are too fundamental to duplicate or translate. Examples: a shared identity/user module between authentication and authorization contexts. Both teams must agree on all changes—high coordination cost. Avoid for loosely related contexts.

Q8. What is the difference between a domain event and an integration event?

Domain event: signals something that happened within a bounded context. Used internally to trigger side effects within the same context. Integration event: published to other bounded contexts via a message broker. Integration events should be schema-stable and versioned. Domain events can change freely.

Q9. How does event storming help with service decomposition?

Event storming is a collaborative workshop where domain experts and developers map all domain events on a timeline. Events naturally cluster around aggregates. Boundaries between clusters become bounded context boundaries. Commands trigger events; policies react to events. The resulting map directly informs service boundaries.

Q10. What is the Customer/Supplier context map pattern?

An upstream (supplier) context provides data/functionality to a downstream (customer) context. The supplier should accommodate customer needs in its roadmap—but has final say on its API. The customer depends on the supplier's published interface. Example: a shared user service (supplier) used by many consuming services (customers).

Q11. What does 'Conformist' mean in context maps?

The downstream conforms entirely to the upstream model with no ability to negotiate changes. Common when integrating with external APIs or legacy systems you don't control. Risk: the upstream's model leaks into your domain. Mitigation: add an ACL on top of the conformist relationship.

Q12. How does Ubiquitous Language prevent integration bugs?

When code uses the same terms as domain experts use in conversation, there's no translation layer between requirements and implementation. Bugs often come from misunderstanding—'order' in the UI means 'purchase order' but code says 'request'. Ubiquitous Language eliminates this. Class names, method names, event names all use exact domain terms.

Q13. What is decomposition by business capability?

Identify stable business capabilities that the organization performs (Order Management, Payment Processing, Customer Management). Each capability becomes a service. Business capabilities are stable—the underlying technology and processes may change, but the capability itself doesn't. This produces durable service boundaries.

Q14. Why is technical-layer decomposition fragile?

Splitting into UI service, service layer, data layer means every new feature requires coordinated changes across all three services. Teams cannot work independently. Every release needs cross-team coordination. This creates a distributed monolith without the benefits of microservices.

Q15. What is an Open-Host Service?

A bounded context that provides a formal, documented, stable API protocol for multiple consumers. The providing team maintains backward compatibility and versioning. Consumers integrate via the published protocol without requiring custom integration. Example: a Payment service with a versioned REST API.

Q16. What is Published Language?

A well-documented interchange format used between bounded contexts. Examples: Avro schemas in a Schema Registry, Protobuf .proto files, OpenAPI YAML. Published Language combined with Open-Host Service creates a stable, discoverable integration point. Schema evolution must maintain compatibility.

Q17. How do aggregates enforce invariants?

The aggregate root contains all business rules for the aggregate cluster. Any operation that changes state goes through the root's methods. The root validates all invariants before allowing the change. Example: Order.addLine() checks that the order is still in DRAFT status before allowing modifications.

Q18. What is the rule about aggregate size?

Keep aggregates small—include only what must be consistent within a single transaction. Large aggregates cause contention (many transactions updating same aggregate) and performance issues. If two concepts don't need to be consistent in the same transaction, they belong in separate aggregates. Reference cross-aggregate by ID, not object reference.

Q19. Explain the 'one aggregate per transaction' rule.

Each database transaction should only modify one aggregate. If a use case seems to need updating two aggregates atomically, use eventual consistency: update first aggregate, publish domain event, second aggregate updates asynchronously. This avoids distributed transactions and keeps aggregates decoupled.

Q20. What is a domain service?

A stateless service that performs domain logic not naturally fitting in any entity or value object. Example: MoneyTransferService.transfer(fromAccount, toAccount, amount)—this operation spans two Account aggregates and doesn't belong in either. Domain services are part of the domain model, not application

services.

Q21. How do you handle cross-aggregate queries?

Options: (1) CQRS with a dedicated read model that denormalizes data from multiple aggregates, (2) Application service joins data from multiple repositories, (3) Domain events update a projection. Never bypass aggregate boundaries by querying the database directly across aggregate tables.

Q22. What is the Separate Ways pattern?

Two bounded contexts have no integration—they develop completely independently. Acceptable when contexts have no meaningful overlap. Teams avoid the coordination overhead of integration. May result in some data duplication, but eliminates coupling. Use when integration cost exceeds duplication cost.

Q23. How do you handle the same concept appearing in multiple bounded contexts?

Each context has its own model of the concept suited to its needs. A 'Product' in the Catalog context has rich descriptions, images, categories. A 'Product' in the Order context has only orderable fields: SKU, price, available stock. Don't create a God object shared across contexts—duplicate and tailor.

Q24. What is event-driven integration between bounded contexts?

Contexts communicate via domain events published to a message broker (Kafka). Events are the Published Language. Receiving contexts translate events to their own model (ACL). Decoupled in time and space. Example: Order context publishes OrderConfirmed; Inventory context subscribes and reserves stock.

Q25. How do you model aggregates in Spring Boot?

Use JPA @Entity for the aggregate root with CascadeType.ALL for child entities. Child entities (OrderLine) should not have their own repository—access always through the root. Use @Embedded for value objects. Domain events collected during aggregate mutations are published after successful persistence.

Q26. What is the difference between application service and domain service?

Domain service: contains domain logic, uses domain concepts, is part of the domain model. Application service: orchestrates use cases, calls domain objects, handles transactions, publishes events, but contains no business logic itself. Application services are thin; domain logic lives in aggregates/domain services.

Q27. How do you prevent model leakage between bounded contexts?

Never share JPA entities or database tables across contexts. Use ACLs for translation. Publish integration events with their own schema (not domain model classes). Use package-private visibility to prevent accidental imports. ArchUnit rules to enforce no cross-context dependencies.

Q28. What is strategic DDD vs tactical DDD?

Strategic DDD: high-level design—bounded contexts, context maps, ubiquitous language. Applicable to entire system architecture. Tactical DDD: implementation patterns—entities, value objects, aggregates, domain services, repositories, factories, domain events. Applied within a single bounded context.

Q29. When is a context map 'Conformist' vs 'ACL'?

Conformist: accept upstream model as-is into your own code—faster, less code, but model leaks in. ACL: wrap all calls through a translation layer—more code, but your domain model stays pure. Choose ACL when the upstream model is messy, unstable, or fundamentally different from your domain concepts.

Q30. Describe how to identify service boundaries using event storming.

Run an event storming session: (1) Post all domain events on a timeline (orange stickies). (2) Add commands that trigger events (blue stickies). (3) Identify aggregates that handle commands (yellow). (4) Mark policy reactions (lilac). (5) Look for natural clusters and pivot points—these become bounded context boundaries and service candidates.

Q31. What makes a good bounded context boundary?

A good boundary has: (1) a single, clear responsibility, (2) terms with unambiguous meaning inside the boundary, (3) a team that owns it end-to-end, (4) minimal integration with other contexts, (5) independent deployability. Bad boundaries split a single business concept across multiple contexts.

Q32. How does DDD relate to microservices?

DDD provides the vocabulary and patterns for decomposing a complex domain. Bounded Contexts map directly to microservice boundaries. Aggregates define transaction boundaries within a service. Domain events enable async integration. Context Maps document service-to-service relationships. DDD is the 'what to build'; microservices is the 'how to deploy it'.

Q33. What is the Published Language pattern with Avro schema?

Define event schemas using Avro IDL registered in Confluent Schema Registry. Producers and consumers use schema IDs in Kafka messages. Schema Registry enforces compatibility rules (BACKWARD: new schema can read old data, FORWARD: old schema can read new data). This is the Published Language—a formal, versioned, enforced interchange format.

Q34. Explain aggregate design for an e-commerce order.

Order aggregate root: Order entity with OrderId, CustomerId (by ID reference), OrderStatus, list of OrderLine value objects, ShippingAddress value object, Money totalAmount. Invariants enforced by Order: can only add lines while DRAFT, total must match sum of lines, cannot ship without confirmed payment. OrderLine is not a separate aggregate—it has no meaning outside an Order.

Q35. How do you handle long-running business processes that span multiple aggregates?

Use Saga pattern: a sequence of local transactions in different aggregates/services coordinated via events. Each step publishes an event; the next service reacts. If a step fails, compensating transactions undo previous steps. Sagas replace distributed transactions (2PC) with eventual consistency.

Q36. What is a factory in DDD?

An object responsible for creating complex aggregates or domain objects when construction logic is too complex for a simple constructor. Keeps creation logic in the domain, not application services. Example: OrderFactory.createFromCart(cart, customer)—builds the Order aggregate with appropriate defaults and validations.

Q37. How do you handle versioning of domain events?

Options: (1) Upcasters: transform old event versions to current format before processing, (2) Schema Registry compatibility rules (BACKWARD/FORWARD), (3) Event versioning with version field in payload, (4) Never delete old event types—always add new versions. Consumer handles multiple versions via type dispatch.

Q38. What is the Repository pattern in DDD?

An abstraction that provides collection-like access to aggregates. Hides persistence details. One repository per aggregate root—never for child entities. Interface in domain layer; implementation in infrastructure. OrderRepository has: findById(OrderId), save(Order), findByCustomerId(CustomerId).

Q39. How do you implement the Repository pattern in Spring Boot?

Define the interface in the domain layer: interface OrderRepository { Optional<Order> findById(OrderId id); void save(Order order); } Implement in infrastructure: @Repository class JpaOrderRepository implements OrderRepository { @Autowired JpaOrderEntityRepository jpa; ... }. This keeps domain layer free of Spring/JPA dependencies.

Q40. What is the difference between a command and an event in CQRS/DDD?

Command: an intent to change state, can be rejected. Imperative mood: PlaceOrder, CancelOrder. Event: a fact that something happened, immutable, cannot be rejected. Past tense: OrderPlaced, OrderCancelled. Commands flow inward (client to service); events flow outward (published to other contexts).

Q41. Explain bounded context integration using REST vs events.

REST (synchronous): Order service calls User service API to get customer data. Simple, immediate. Coupling: if User service is down, Order service fails. Events (asynchronous): User service publishes CustomerUpdated event; Order service subscribes and caches customer data. Decoupled availability, but eventual consistency. Choose based on whether you need real-time consistency.

Q42. What is the Open/Closed principle applied to bounded contexts?

A bounded context should be open for extension (add new features/events) but closed for modification (never break existing API consumers). New event types can be added. Existing events must remain backward compatible. Published Language schemas must evolve additively—add fields with defaults, never remove required fields.

Q43. How do you handle a 'God Service' anti-pattern?

A service that has too many responsibilities (everything about 'user': authentication, profile, preferences, social graph, payment methods). Split by subdomain: Authentication service, User Profile service, Social Graph service. Each becomes its own bounded context with its own model. Decompose when a service has multiple teams working on it simultaneously.

Q44. What are the signs of poorly defined bounded contexts?

Signs: (1) Teams stepping on each other's changes, (2) Same database table used by multiple services, (3) A change in one service requires simultaneous changes in others, (4) The same concept (e.g., 'product') has wildly different data in different places, (5) Services call each other in long synchronous chains.

Q45. How do you migrate an existing monolith to DDD bounded contexts?

(1) Run event storming to discover domain events and natural boundaries. (2) Identify existing modules that map to bounded contexts. (3) Add package-private + ArchUnit boundaries to enforce context isolation. (4) Replace cross-context direct calls with domain events or ACL adapters. (5) Extract high-priority contexts to separate services using Strangler Fig.

Q46. What is the role of the Ubiquitous Language in code?

Class names, method names, variable names, event names all use exact domain terms as used by business experts. If a business expert says 'place order', the code has placeOrder(). If they say 'fulfill shipment', the code has fulfillShipment(). This eliminates translation between requirements and implementation, making bugs easier to trace to business rules.

Q47. What is a specification pattern?

An encapsulation of a business rule that determines whether a domain object satisfies certain criteria. Can be combined with AND/OR/NOT. Example: EligibleForDiscountSpecification implements Specification<Customer>. Keeps complex query logic in the domain layer rather than scattered in repositories or services.

Q48. Describe the bounded context for a multi-tenant SaaS application.

Typical bounded contexts: Identity (authentication, authorization, tenants), Organization (tenant management, user membership), Subscription/Billing (plans, invoices, payment), Product/Feature (what the product does). Each context has its own schema with tenant_id column for data isolation. Context Map: Identity is Open-Host Service; others use it via C/S relationship.

Q49. What is eventual consistency between bounded contexts?

When contexts communicate via events, the receiving context's data is not immediately consistent with the source context. Example: User updates email in User context; Order context receives CustomerEmailUpdated event and updates its local copy. During the lag, Order context may show stale email. This is a design trade-off: decoupled availability vs. immediate consistency.

Q50. How do you test bounded context integration?

Consumer-driven contract tests (Pact): Consumer defines expected event schemas and API contracts. Provider verifies contracts in CI. This ensures bounded context integration doesn't break without spinning up all services. Integration tests use Testcontainers for Kafka and real DBs.